

CM 8A - LA PROGRAMMATION ORIENTÉE OBJET

PROGRAMMATION PYTHON – MASTER 1 MAS

Romain Tavenard

2026

✉ romain.tavenard@univ-rennes2.fr



VOCABULAIRE



```
class MaClasseTropClasse:
    def __init__(self):
        self.a = 123

o = MaClasseTropClasse()
```

Que vaut `o.a` ici ?

```
class MaClasseTropClasse:
    def __init__(self, x):
        self.a = 123 + x

o = MaClasseTropClasse(12)
```

Que vaut `o.a` ici ?

```
class MaClasseTropClasse:
    def __init__(self):
        self.a = 123          # Propriété (== attribut)

    def f(self, b):           # Méthode
        return b + self.a - 4

    @property
    def double_de_a(self):    # Propriété (calculée)
        return 2 * self.a

o = MaClasseTropClasse()
print(o.a)
print(o.f(0))
print(o.double_de_a)
```

“ Implémentez une classe *NombreComplexe* telle que l'on puisse exécuter le code suivant :

```
c0 = NombreComplexe(10, 0) # 10
c1 = NombreComplexe(0, 5)  # 5i
c2 = NombreComplexe(3, -2) # 3 - 2i
print(c1.module())         # Affiche 5
print(c0.distance(c1))     # Affiche 11.18...
```

”

SURCHARGE D'OPÉRATEUR



Que se passe-t-il lorsqu'on écrit `a + b` en Python ?

Que se passe-t-il lorsqu'on écrit `a + b` en Python ?

L'interpréteur...

1. recherche la classe de `a`
2. examine cette classe pour trouver une méthode spéciale `__add__`
3. appelle cette méthode, comme si on avait écrit `a.__add__(b)`

```
class MaClasseTropClasse:
    def __init__(self, x=0):
        self.a = 123 + x

    def __add__(self, autre):
        return self.a + autre.a

o1 = MaClasseTropClasse()
o2 = MaClasseTropClasse(10)
```

Que vaut `o1 + o2` ici ?

“ Reprenez votre classe `NombreComplexe`
et implémentez l'opérateur d'addition :

```
c0 = NombreComplexe(10, 0) # 10  
c1 = NombreComplexe(0, 5) # 5i  
c2 = c0 + c1                # c2 doit être de classe NombreComplexe
```

”

À quoi servent les opérateurs suivants ?

1. `__mul__`

À quoi servent les opérateurs suivants ?

1. `__mul__`
2. `__sub__`

À quoi servent les opérateurs suivants ?

1. `__mul__`
2. `__sub__`
3. `__truediv__` / `__floordiv__`

À quoi servent les opérateurs suivants ?

1. `__mul__`
2. `__sub__`
3. `__truediv__` / `__floordiv__`
4. `__repr__`